

Solution 1

Solution 1 (a)

Online Algorithm Implementation: Mean

```

1  """
2  Implementation of online algo from class
3  read data-1.txt using np.loadtxt(data-1.txt)
4  """
5  # load libraries
6  import numpy as np
7  import matplotlib.pyplot as plt
8  import seaborn as sns
9
10 # quick check data-1.txt has 5000 pts
11 # cat data-1.txt | wc -l -> 5000
12
13 # load data-1.txt
14 data = np.loadtxt('data-1.txt')
15
16 def online_algorithm_mean(data):
17     mean_i = []
18     dt = []
19     current_mean = 0
20     for t,x_t in enumerate(data):
21         dt.append(t+1)
22         current_mean = current_mean + ((x_t-current_mean)/(t+1))
23         mean_i.append(current_mean)
24     print(f"final mean: {mean_i[-1]}")
25     return dt, mean_i
26
27 def plot_mean_dt(dt,mean_i,true_mean):
28     fig,ax = plt.subplots(nrows=1,ncols=1,figsize=(8,10))
29     sns.lineplot(x=dt,y=mean_i,markers='o',ax=ax,color='blue',label='OA')
30     sns.lineplot(x=dt,y=[true_mean for i in
31         dt],markers='--',ax=ax,color='red',label='True Mean')
32     ax.set_xlabel('dt')
33     ax.set_ylabel('mean')
34     ax.set_ylim(min(mean_i)-1e5,true_mean+1e5)
35     print(min(mean_i))
36     title = f"Online Algorithm(OA) for Mean\n True Mean: {true_mean:.3f} \n OA Mean:
37         {mean_i[-1]:.3f}"
38     ax.set_title(title)
39     fig.savefig('hw3_q1.png')
40
41 # calc true mean and print it
42 true_mean = sum(data)/len(data)
43 print(true_mean)
44
45 dt, mean_i = online_algorithm_mean(data)
46 plot_mean_dt(dt,mean_i,true_mean)

```

Comments

The true mean is 2,991,834.673 and is identical to the final mean calculated via `online_algorithm_mean(data)` function.

Solution 1

Solution 1 (b)

Plot

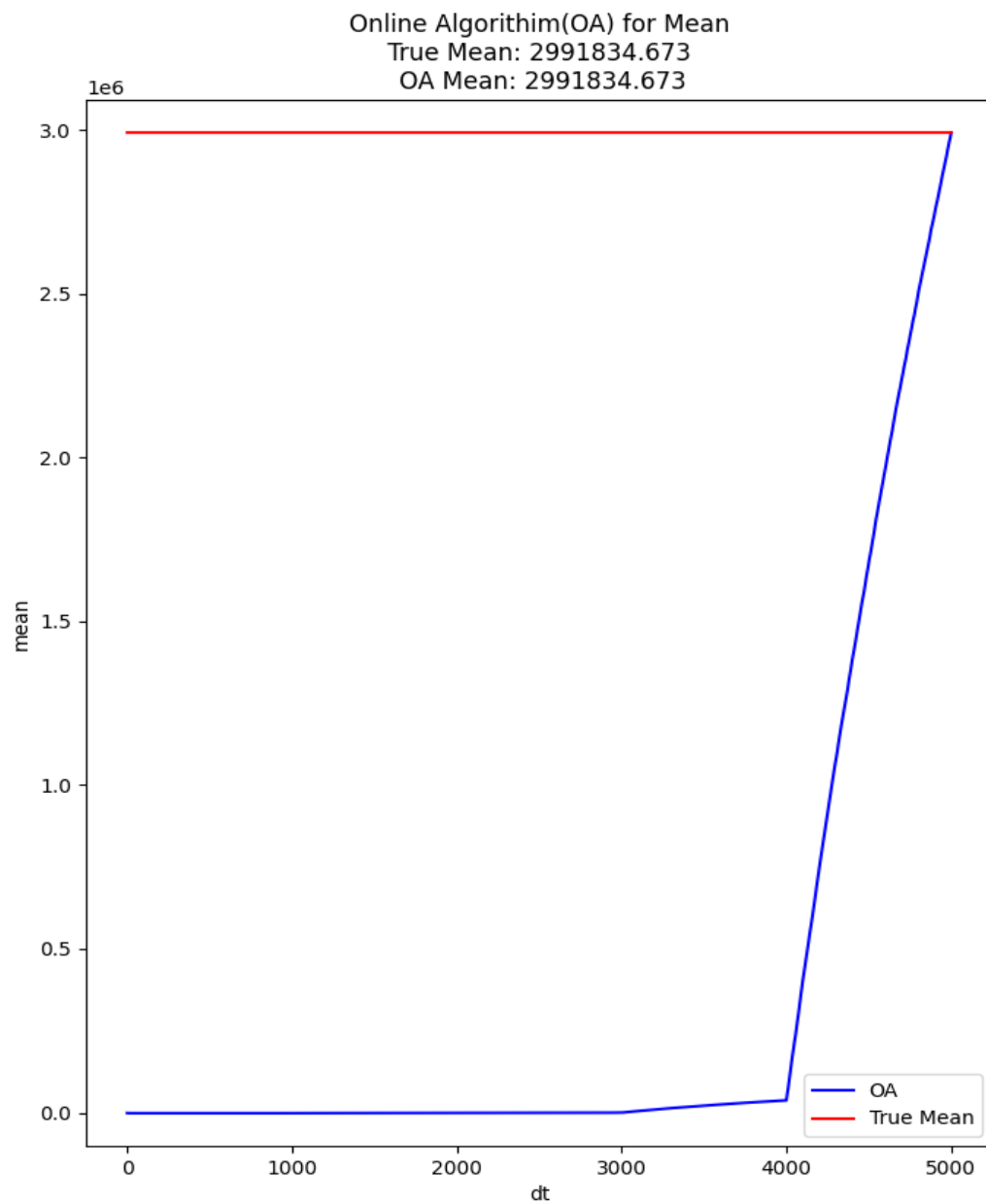


Figure 1: Figure shows all 5000 intermediate values of mean

Solution 2

Solution 2 (a)**Pseudocode**

Algorithm 1 Online Variance Calculation

Initialize:

$t \leftarrow 0$	▷ Count of data points
$\mu \leftarrow 0.0$	▷ Running mean of X
$M_2 \leftarrow 0.0$	▷ Running mean of X^2
$v \leftarrow 0.0$	▷ Running variance

For each new data point x_t :

$t \leftarrow t + 1$	
$\mu \leftarrow \mu + (x_t - \mu)/t$	▷ Update mean of X
$M_2 \leftarrow M_2 + (x_t^2 - M_2)/t$	▷ Update mean of X^2
$v \leftarrow M_2 - \mu^2$	▷ Calculate current variance

Solution 2

Solution 2 (b)

Online Algorithm Implementation: Variance

```
1  # load libraries
2  import numpy as np
3
4  # load data-1.txt
5  data = np.loadtxt('data-1.txt')
6
7  def online_algorithm_variance(data):
8      mean_x = 0.0
9      mean_x_squared = 0.0
10     variances = []
11
12     for t,x_t in enumerate(data):
13         # Update the mean of x
14         mean_x = mean_x + ((x_t-mean_x)/(t+1))
15         # Update the mean of x-squared
16         mean_x_squared = mean_x_squared + ((x_t**2-mean_x_squared)/(t+1))
17
18         variances.append(mean_x_squared - mean_x**2)
19
20     print(f"final variance: {variances[-1]:.3f}")
21     return variances
22
23 # calc true variance and print
24 true_variance = np.var(data)
25 print(f"true variance: {true_variance:.3f}")
26
27 # calc online_algorithm_variance
28 variances = online_algorithm_variance(data)
```

Comments

The true variance using numpy: 36,618,966,207,041.938 while the variance calculated via `online_algorithm_variance(data)` was: 36,618,966,207,041.961. This discrepancy between the true and *online variance calculation* arises because the online formula is sensitive to floating point precision errors, especially when subtracting two large, similar numbers.

Solution 3

Solution 3 (a)**True Median**

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 """
4 Implementation of online algo from class
5 read data-1.txt using np.loadtxt(data-1.txt)
6 """
7 data = np.loadtxt
```

ANSWER THIS: *report true median of data-1.txt*

Solution 3

Solution 3 (b)**Random Sample with Replacement Algorithm Implementation: Median**

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 """
4 Implementation of online algo from class
5 read data-1.txt using np.loadtxt(data-1.txt)
6 """
7 data = np.loadtxt
```

ANSWER THIS: *report true median of data-1.txt*

Solution 3

Solution 3 (c)**Random Sample with Replacement Algorithm Implementation: Median(sample: 500)**just repeat (b) with $n = 500$

Solution 4

something something something

Solution 5

Solution 5 (a)

something something something

Solution 5

Solution 5 (b)

something something something

Solution 6

Solution 6 (a)

something something something

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 """
4 code different
5 """
6 data = np.loadtxt
```

Solution 6

Solution 6 (b)

something something something

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 """
4 code different
5 """
6 data = np.loadtxt
```

Solution 7

Solution 7 (a)

something something something

Solution 7

Solution 7 (b)

something something something
